PROCEDIMENTO OPERACIONAL PADRÃO · INFRAESTRUTURA DE TI

GLPI 12

Implantação e recuperação operacional

Guia validável, do sistema operacional à operação, com correções baseadas no erro HTTP 500 observado.

PLATAFORMAS
Debian 13 · Ubuntu Server LTS

COMPONENTES
Apache · PHP-FPM · MariaDB

REVISÃO
3.0 · 28 de outubro de 2026

CONTROLE
POP-GLPI-DEPLOY-003

TRILHA A Instalação nova **TRILHA B** Correção do servidor existente
TRILHA C Validação, backup e aceite corporativo

Índice do procedimento

Consulte a Trilha A apenas em VM nova; use a Trilha B se a instância já existir.

TRILHA A — INSTALAÇÃO LIMPA

- A00 — Escolhas antes da instalação
- A01 — Preparar Debian/Ubuntu
- A02 — Instalar a pilha e corrigir PATH
- A03 — Criar MariaDB isolado
- A04 — Baixar o GLPI 12.0.1
- A05 — Separar código, configurações, dados e logs
- A06 — Ajustar PHP-FPM e PHP CLI
- A07 — Integrar Apache ao PHP-FPM
- A08 — Configurar o VirtualHost do GLPI
- A09 — Verificar requisitos e concluir o instalador
- A10 — Configurar tarefas automáticas
- A11 — Requisitos de segurança para produção
- A12 — Backup, restauração e monitoramento

TRILHA B — CORREÇÃO DO GLPI JÁ INSTALADO

- B00 — Evidências reais do servidor e limites
- B01 — Diagnóstico do erro 500 antes de alterar arquivos
- B02 — Corrigir mkdir(): Permission denied com segurança
- B03 — Correção validada: cache Twig com caminho incorreto
- B04 — Evitar as falhas de comandos encontradas

TRILHA C — HOMOLOGAÇÃO, OPERAÇÃO E ACEITE

- C01 — Testes funcionais e de recuperação
- C02 — Checkpoints GO / NO-GO
- C03 — Registro de execução
- C04 — Referências oficiais e notas de controle

Manual técnico operacional | Revisão 3.0 | 28/10/2026
Plataformas: Debian 13 e Ubuntu Server LTS compatível (24.04 e 26.04 mediante validação do PHP).
Pilha: Apache 2.4 + PHP-FPM + MariaDB + GLPI 12.0.1.
Escopo: GLPI ponta a ponta, sem implantação de DNS público, AD ou SMTP.

> **Regra de segurança:** a trilha A é **somente para servidor novo, sem GLPI instalado**. A trilha B é para diagnosticar e corrigir o GLPI **já instalado**. Nunca execute os comandos de criação de banco ou extração de arquivos da trilha A em produção existente.

> **Critério de sucesso:** "100%" significa **100% dos testes de aceite aplicáveis aprovados**, e não garantia de disponibilidade absoluta. Um acesso bem-sucedido à tela de login não comprova backup, segurança, notificações, inventário, funcionamento futuro de plugins nem restauração.

Controle documental

Campo	Informação
Identificador	POP-GLPI-DEPLOY-003
Base de referência	GLPI 12.0.1 (release de segurança de 28/10/2026)
Sistemas	Debian 13 / Ubuntu Server LTS
Servidor observado nos logs	`GLPI`, IP de teste `192.168.18.150`
Estado informado pelo operador	Acesso voltou a funcionar após reconstrução do cache
Estado ainda a comprovar	HTTPS, backup/restauração, cron e testes funcionais completos
Responsável pela execução	_____
Mudança / chamado	_____
Homologação / aceite	_____

Como executar este manual

Cada bloco de código indica sua linguagem. **Bash** deve ser executado no terminal Linux; **SQL** deve ser executado dentro do console MariaDB; **PHP/Apache** é conteúdo para um arquivo aberto no `nano`, não um comando de terminal. Os resultados esperados são instruções de leitura, não texto para copiar.

As credenciais de exemplo são marcadores: substitua por senhas geradas e mantidas no cofre corporativo. Execute cada bloco separadamente, valide a saída e interrompa ao encontrar divergência relevante.

TRILHA A — INSTALAÇÃO LIMPA

A00 — Escolhas antes da instalação

A00.01 — Confirmar a versão. Em 28/10/2026, GLPI **12.0.1** é uma versão estável atual com correções de segurança. O GLPI 12 requer **PHP 8.3 ou posterior** e **MariaDB 10.11 ou posterior**. Não presuma que todos os plugins utilizados no GLPI 11 são compatíveis com o 12.

A00.02 — Planejamento. Para ambiente inicial pequeno, estime 2–4 vCPU, 4–8 GB RAM e disco conforme volume de inventário/anexos; **não são requisitos mínimos oficiais**. Planeje IP estável, relógio sincronizado, acesso administrativo restrito, monitoramento, backup fora da VM e rollback por mudança autorizada.

A00.03 — Padrão de pastas.

Caminho	Uso	Gravação pelo serviço web
`/var/www/glpi`	Código e dependências	Não, em operação normal
`/var/www/glpi/public`	Única raiz publicada pelo Apache	Não
`/etc/glpi`	Configuração e chaves	Sim durante instalação; restringir depois
`/var/lib/glpi/files`	Documentos, sessões, cache e dados	Sim
`/var/log/glpi`	Logs da aplicação	Sim
`/var/lib/glpi/plugins`	Marketplace	Sim, se habilitado

A00.04 — Separação de contexto. Use `192.168.18.150` **apenas como exemplo de laboratório**. No ambiente corporativo, substitua pelo IP definitivo. A instalação pode ser validada provisoriamente via HTTP em rede isolada, mas a liberação de produção exige TLS confiável.

A01 — Preparar Debian/Ubuntu

A01.01 — Tornar-se administrador. Se já estiver no usuário `root`, pule este comando.

```
sudo -i
```

Verificar: o prompt termina em `#`. Nunca copie `root@GLPI:/tmp#` como parte de um comando.

A01.02 — Identificar o sistema.

```
cat /etc/os-release
```

Verificar: Debian `VERSION_ID=13`, ou uma versão Ubuntu LTS aprovada pelo projeto.

A01.03 — Verificar endereço IPv4.

```
ip -br -4 address
```

Verificar: anote o IP da placa de rede de produção; não o confunda com `127.0.0.1`.

A01.04 — Verificar memória.

```
free -h
```

A01.05 — Verificar espaço disponível.

```
df -h /
```

A01.06 — Verificar sincronismo de horário.

```
timedatectl status
```

Verificar: relógio/NTP corretos. Ajuste a política corporativa antes de prosseguir.

A01.07 — Atualizar índices do APT.

```
apt update
```

A01.08 — Aplicar atualizações na VM NOVA. Planeje reinicialização se necessário.

```
apt upgrade -y
```

A02 — Instalar a pilha e corrigir PATH

A02.01 — Instalar Apache, MariaDB e PHP-FPM dos repositórios oficiais. Não inclua `php-imap`: esse pacote pode não estar disponível, e não é necessário para o funcionamento básico do GLPI.

```
apt install -y apache2 mariadb-server mariadb-client php php-cli php-fpm \
  php-mysql php-curl php-gd php-intl php-mbstring php-bcmath \
  php-xml php-zip php-bz2 php-ldap php-opcache php-apcu \
  php-soap curl wget ca-certificates openssl tar unzip cron \
  logrotate sudo
```

Verificar: o APT termina sem erro. Se houver dependência indisponível, não avance até identificar qual pacote falhou.

A02.02 — Descobrir a versão PHP da instalação. Essa variável existe apenas na sessão atual e deve ser refeita após novo login SSH.

```
PHPVER=$(php -r 'echo PHP_MAJOR_VERSION.".".PHP_MINOR_VERSION;')
```

A02.03 — Conferir o valor.

```
echo "$PHPVER"
```

Resultado esperado: Debian 13 usualmente `8.4`; no Ubuntu LTS pode ser outra versão. Confirme os requisitos do GLPI antes de prosseguir.

A02.04 — Verificar versão PHP real.

```
php -v
```

A02.05 — Corrigir `a2enmod: command not found`. Os utilitários do Apache residem em `/usr/sbin`. Primeiro, visualize o PATH:

```
printf '%s\n' "$PATH"
```

A02.06 — Corrigir PATH na sessão atual, se necessário. Não modifica permanentemente o servidor.

```
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
```

A02.07 — Confirmar `a2enmod`.

```
command -v a2enmod
```

Resultado esperado: `/usr/sbin/a2enmod`.

A02.08 — Confirmar `a2enconf`.

```
command -v a2enconf
```

Resultado esperado: `/usr/sbin/a2enconf`. Para futuros logins use `su -` ou `sudo -i`, evitando sessões root sem PATH administrativo.

A02.09 — Iniciar serviços.

```
systemctl enable --now apache2 mariadb "php${PHPVER}-fpm" cron
```

A02.10 — Verificar Apache.

```
systemctl is-active apache2
```

A02.11 — Verificar MariaDB.

```
systemctl is-active mariadb
```

A02.12 — Verificar PHP-FPM.

```
systemctl is-active "php${PHPVER}-fpm"
```

Resultado esperado dos três últimos: `active`.

A02.13 — Conferir extensões PHP.

```
php -m | grep -Ei 'bcmath|curl|gd|intl|ldap|mbstring|mysqli|openssl|xml|zip'
```

Verificar: os módulos obrigatórios estão presentes; o verificador do GLPI fará a validação completa.

A03 — Criar MariaDB isolado

> **Somente instalação limpa.** Não execute `CREATE DATABASE`, `CREATE USER` ou `db:install` contra um banco que já possui GLPI. O banco e usuário devem existir apenas uma vez.

A03.01 — Entrar no console administrativo.

```
mariadb
```

A partir daqui o prompt será do MariaDB, não do Bash.

A03.02 — Criar banco UTF8MB4.

```
CREATE DATABASE glpidb CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

A03.03 — Criar usuário dedicado. Substitua `SENHA_ALEATORIA_DO_COFRE` por senha segura **antes** de executar.

```
CREATE USER 'glpi'@'localhost' IDENTIFIED BY 'SENHA_ALEATORIA_DO_COFRE';
```

A03.04 — Permitir acesso apenas ao banco GLPI.

```
GRANT ALL PRIVILEGES ON glpidb.* TO 'glpi'@'localhost';
```

A03.05 — Aplicar as permissões.

```
FLUSH PRIVILEGES;
```

A03.06 — Listar os bancos.

```
SHOW DATABASES;
```

Verificar: `glpidb` consta na lista, e nenhuma mensagem de erro ocorreu.

A03.07 — Sair do MariaDB.

```
EXIT;
```

A03.08 — Testar a conta do GLPI; a senha será solicitada.

```
mariadb -u glpi -p glpidb
```

Depois de confirmar acesso, digite `EXIT;` no console MariaDB.

A03.09 — Verificar o endereço de escuta do banco. Em VM com banco local, não há motivo para expor MariaDB à rede.

```
mariadb -e "SHOW VARIABLES LIKE 'bind_address';"
```

Verificar: preferencialmente escuta local, com firewall restringindo a porta 3306.

A04 — Baixar o GLPI 12.0.1

A04.01 — Certificar-se de que o diretório ainda não existe. Se aparecer uma instalação existente, **pare**; não extraia sobre ela.

```
ls -ld /var/www/glpi
```

Resultado esperado: erro `No such file or directory` em uma VM recém-provisionada.

A04.02 — Entrar no diretório temporário.

```
cd /tmp
```

A04.03 — Baixar o arquivo de release oficial.

```
wget https://github.com/glpi-project/glpi/releases/download/12.0.1/glpi-12.0.1.tgz
```

A04.04 — Registrar o hash local. Compare com hash/assinatura confiável se o projeto os disponibilizar. O hash isolado não prova autenticidade.

```
sha256sum /tmp/glpi-12.0.1.tgz
```

A04.05 — Verificar se o arquivo pode ser lido.

```
tar -tzf /tmp/glpi-12.0.1.tgz | head
```

Verificar: os arquivos estão dentro de `glpi/`.

A04.06 — Extrair somente na VM nova.

```
tar -xzf /tmp/glpi-12.0.1.tgz -C /var/www
```

A04.07 — Conferir console GLPI.

```
test -f /var/www/glpi/bin/console && echo 'Console GLPI presente'
```

A04.08 — Conferir a raiz pública.

```
test -f /var/www/glpi/public/index.php && echo 'Diretorio public presente'
```

A05 — Separar código, configurações, dados e logs

Esse é o modelo recomendado para manutenção segura. **Configuração, dados e cache não são pastas de bibliotecas PHP**. Não copie `/var/www/glpi/vendor` para `/var/lib/glpi/vendor`.

A05.01 — Criar configuração e dados.

```
install -d -o www-data -g www-data -m 0750 /etc/glpi /var/lib/glpi/files
```

A05.02 — Criar diretórios de logs e Marketplace.

```
install -d -o www-data -g www-data -m 0750 /var/log/glpi /var/lib/glpi/plugins
```

A05.03 — Copiar arquivos-base de configuração.

```
cp -a /var/www/glpi/config/. /etc/glpi/
```

A05.04 — Copiar arquivos de dados iniciais.

```
cp -a /var/www/glpi/files/. /var/lib/glpi/files/
```

A05.05 — Copiar arquivos iniciais do Marketplace (se existentes).

```
cp -a /var/www/glpi/marketplace/. /var/lib/glpi/plugins/
```

A05.06 — Verificar arquivo de redirecionamento existente.

```
ls -l /var/www/glpi/inc/downstream.php
```

Se já existir, inspecione e preserve as definições originais em vez de sobrescrevê-las cegamente.

A05.07 — Abrir o arquivo de redirecionamento (se for necessário criá-lo).

```
nano /var/www/glpi/inc/downstream.php
```

A05.08 — Conteúdo para o arquivo `downstream.php` (PHP; NÃO executar no Bash).

```
<?php
define('GLPI_CONFIG_DIR', '/etc/glpi/');
if (file_exists(GLPI_CONFIG_DIR . '/local_define.php')) {
    require_once GLPI_CONFIG_DIR . '/local_define.php';
}
```

Verificar: o caminho de configuração agora aponta para `/etc/glpi`.

A05.09 — Abrir `local_define.php`.

```
nano /etc/glpi/local_define.php
```

A05.10 — Conteúdo para `local_define.php` (PHP; NÃO executar no Bash).

```
<?php
define('GLPI_VAR_DIR', '/var/lib/glpi/files');
define('GLPI_LOG_DIR', '/var/log/glpi');
define('GLPI_MARKETPLACE_DIR', '/var/lib/glpi/plugins');
```

Verificar: o diretório de cache passará a ser `/var/lib/glpi/files/_cache`; as bibliotecas `vendor` permanecem no diretório de código `/var/www/glpi/vendor`.

A05.11 — Tornar a árvore de código pertencente ao administrador. Em instalação nova, antes de personalizar arquivos.

```
chown -R root:root /var/www/glpi
```

A05.12 — Permissões dos diretórios do código.

```
find /var/www/glpi -type d -exec chmod 0755 {} +
```

A05.13 — Permissões dos arquivos do código.

```
find /var/www/glpi -type f -exec chmod 0644 {} +
```

A05.14 — Garantir propriedade dos diretórios graváveis.

```
chown -R www-data:www-data /etc/glpi /var/lib/glpi /var/log/glpi
```

A05.15 — Proteger a pasta de configuração.

```
chmod 0750 /etc/glpi
```

A05.16 — Proteger arquivos de configuração.

```
find /etc/glpi -type f -exec chmod 0640 {} +
```

A05.17 — Verificar escrita em configuração como usuário web.

```
sudo -u www-data test -w /etc/glpi && echo 'Escrita em /etc/glpi OK'
```

A05.18 — Verificar escrita nos dados.

```
sudo -u www-data test -w /var/lib/glpi/files && echo 'Dados gravaveis OK'
```

A05.19 — Verificar escrita nos logs.

```
sudo -u www-data test -w /var/log/glpi && echo 'Logs gravaveis OK'
```

Cuidado: não use `chmod -R 777`. Após a instalação, é possível restringir ainda mais `/etc/glpi` conforme o modelo de operação, preservando a capacidade de acesso às chaves.

A06 — Ajustar PHP-FPM e PHP CLI

PHP-FPM atende as requisições do navegador; PHP CLI executa `bin/console` e o CRON. São **arquivos `php.ini` diferentes**. As alterações de fuso horário, memória e limites de upload precisam ser coerentes.

A06.01 — Abrir a configuração do PHP-FPM. Refaça a variável `PHPVER` se iniciou nova sessão SSH.

```
nano "/etc/php/${PHPVER}/fpm/php.ini"
```

A06.02 — Localizar/ajustar diretivas existentes no arquivo (NÃO executar no terminal). Valores são exemplos iniciais para homologação e precisam de dimensionamento.

```
memory_limit = 512M
upload_max_filesize = 64M
post_max_size = 72M
max_execution_time = 300
date.timezone = America/Sao_Paulo
session.cookie_httponly = On
session.cookie_samesite = Lax
```

Use `Ctrl + W` no `nano` para localizar cada chave; `Ctrl + O`, `Enter`, `Ctrl + X` para salvar. **Não ative `session.cookie_secure = On` enquanto o servidor estiver disponível apenas por HTTP**, pois prejudicará a sessão. Ative após HTTPS válido.

A06.03 — Abrir configuração do CLI.

```
nano "/etc/php/${PHPVER}/cli/php.ini"
```

A06.04 — Localizar/ajustar no CLI (NÃO executar no terminal).

```
memory_limit = 512M
date.timezone = America/Sao_Paulo
```

A06.05 — Reiniciar PHP-FPM.

```
systemctl restart "php${PHPVER}-fpm"
```

A06.06 — Confirmar PHP-FPM ativo.

```
systemctl is-active "php${PHPVER}-fpm"
```

A06.07 — Conferir o fuso horário no CLI.

```
php -r 'echo date_default_timezone_get(), PHP_EOL;'
```

Verificar: `America/Sao_Paulo` quando essa for a política de fuso da empresa; outras empresas devem usar o fuso oficial de operação.

A07 — Integrar Apache ao PHP-FPM

A07.01 — Habilitar os módulos completos. Na sua instalação anterior, usar `a2enmod` sem o caminho completo falhou devido ao PATH. O comando com `/usr/sbin/` independe dessa variável.

```
/usr/sbin/a2enmod rewrite proxy_fcgi setenvif headers
```

Cuidado: não adicione ponto depois de `a2enmod` (`a2enmod.` está errado). Se aparecer `Which module(s) do you want to enable?`, use `Ctrl+C` e execute a linha completa com os quatro nomes.

A07.02 — Verificar se o módulo PHP clássico está habilitado.

```
/usr/sbin/apache2ctl -M | grep php_module
```

Verificar: se houver `php_module`, você está com `mod_php`. Somente em VM **exclusiva do GLPI**, no projeto PHP-FPM, faça a próxima etapa. Em servidor compartilhado, avalie outros sites.

A07.03 — Desabilitar mod_php somente se o teste acima retornar `php_module`.

```
/usr/sbin/a2dismod "php${PHPVER}"
```

A07.04 — Confirmar que existe a configuração FPM para Apache.

```
test -f "/etc/apache2/conf-available/php${PHPVER}-fpm.conf" && echo 'Configuracao FPM presente'
```

A07.05 — Habilitar a integração FPM.

```
/usr/sbin/a2enconf "php${PHPVER}-fpm"
```

A07.06 — Verificar o link ativado no Apache.

```
ls -l "/etc/apache2/conf-enabled/php${PHPVER}-fpm.conf"
```

A07.07 — Testar a sintaxe.

```
/usr/sbin/apache2ctl configtest
```

Resultado esperado: `Syntax OK`. A mensagem `AH00558` sobre o nome do servidor é aviso, não a causa de erro HTTP 500.

A07.08 — Reiniciar o Apache, somente com sintaxe válida.

```
systemctl restart apache2
```

A07.09 — Confirmar os módulos.

```
/usr/sbin/apache2ctl -M | grep -E 'rewrite|proxy_fcgi|setenvif|headers'
```

Resultado esperado: `rewrite_module`, `proxy_fcgi_module`, `setenvif_module` e `headers_module`.

A07.10 — Confirmar que `php_module` não foi mantido, quando FPM for o padrão.

```
/usr/sbin/apache2ctl -M | grep php_module
```

Resultado esperado nesse desenho: nenhuma saída. *A simples condição `php-fpm active` não comprova que o Apache realmente utiliza FPM.*

A08 — Configurar o VirtualHost do GLPI

A08.01 — Abrir arquivo do site.

```
nano /etc/apache2/sites-available/glpi.conf
```

A08.02 — Inserir o conteúdo do VirtualHost (arquivo Apache; NÃO executar no Bash). Troque o IP fictício pelo IP de homologação da VM. A raiz pública é obrigatoriamente `/var/www/glpi/public`.

```
<VirtualHost *:80>
    ServerName 192.168.18.150
    DocumentRoot /var/www/glpi/public

    <Directory /var/www/glpi/public>
        Options FollowSymLinks
        AllowOverride None
        Require all granted
        RewriteEngine On
        RewriteCond %{HTTP:Authorization} ^(.+)$
        RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
        RewriteCond %{REQUEST_FILENAME} !-f
        RewriteRule ^(.*)$ index.php [QSA,L]
    </Directory>

    ErrorLog ${APACHE_LOG_DIR}/glpi_error.log
    CustomLog ${APACHE_LOG_DIR}/glpi_access.log combined
</VirtualHost>
```

A08.03 — Habilitar somente o site GLPI.

```
/usr/sbin/a2ensite glpi.conf
```

A08.04 — Desabilitar o site padrão, somente em VM dedicada.

```
/usr/sbin/a2dissite 000-default.conf
```

A08.05 — Verificar sintaxe antes de aplicar.

```
/usr/sbin/apache2ctl configtest
```

A08.06 — Aplicar configuração válida.

```
systemctl reload apache2
```

A08.07 — Verificar VirtualHosts habilitados.

```
/usr/sbin/apache2ctl -S
```

Verificar: `glpi.conf` responde na porta esperada e o DocumentRoot é `/var/www/glpi/public`.

A08.08 — Testar requisição HTTP local.

```
curl -I http://127.0.0.1
```

Resultado esperado: HTTP `200` ou redirecionamento/instalador válido. HTTP `500` exige diagnóstico na Trilha B antes de prosseguir.

A09 — Verificar requisitos e concluir o instalador

A09.01 — Entrar no diretório de aplicação.

```
cd /var/www/glpi
```

A09.02 — Listar os comandos do GLPI 12.

```
sudo -u www-data php bin/console list
```

Verificar: deve aparecer `system:check_requirements`. Em seu servidor, o prefixo `glpi:system:` não foi reconhecido; por isso usamos o nome listado efetivamente no console.

A09.03 — Executar a checagem oficial.

```
sudo -u www-data php bin/console system:check_requirements
```

Critério de avanço: todos os itens marcados `[REQUIRED]` precisam estar `[OK]`. `SKIPPED` pode significar "não aplicável". Corrija erros de `glpicrypt.key`, `oauth.pem` e `oauth.pub` **na pasta de configuração realmente usada**.

A09.04 — Conferir os diretórios usados nesta arquitetura.

```
ls -ld /etc/glpi /var/lib/glpi/files /var/log/glpi
```

A09.05 — Abrir o assistente no navegador de homologação. Digite `http://IP_DA_VM` em uma máquina autorizada. Não copie `IP_DA_VM` literalmente como endereço. Escolha **Português (Brasil)**, **Instalar**, e aceite os termos.

A09.06 — Configurar a ligação com MariaDB. No assistente, informe `localhost`, usuário de banco `glpi` e a senha SQL exclusiva criada na etapa A03. Selecione o banco **glpidb**. Se o banco já contiver tabelas, interrompa: isso pode ser outra instalação.

A09.07 — Concluir o primeiro login. Se a instalação fornecer o usuário `glpi` com senha padrão `glpi`, use apenas para o acesso inicial e **troque imediatamente a senha**. Verifique e desative/altere outras contas padrão existentes, sem remover o último superadministrador. Defina um administrador nominal e os métodos de autenticação aprovados.

A09.08 — Validar novamente depois da instalação.

```
sudo -u www-data php /var/www/glpi/bin/console system:check_requirements
```

A09.09 — Conferir estado da aplicação.

```
sudo -u www-data php /var/www/glpi/bin/console system:status
```

Cuidado: `system:status` lista o status técnico, mas não substitui o teste manual de abertura de chamado, anexo, notificações ou backup.

A10 — Configurar tarefas automáticas

A documentação do GLPI recomenda execução do `front/cron.php` **a cada minuto** pelo usuário do servidor web. A execução automática é necessária para tarefas agendadas, filas e notificações. Antes de mudar um servidor já em produção, verifique se a linha não existe para evitar duplicidade.

A10.01 — Conferir o cron do www-data.

```
crontab -u www-data -l
```

A10.02 — Editar o cron apenas se a entrada ainda não existir.

```
crontab -u www-data -e
```

A10.03 — Inserir uma única linha (conteúdo do crontab).

```
* * * * * /usr/bin/php /var/www/glpi/front/cron.php >> /var/log/glpi/cron.log 2>&1
```

A10.04 — Conferir a entrada salva.

```
crontab -u www-data -l
```

A10.05 — Executar uma vez manualmente, para diagnóstico.

```
sudo -u www-data php /var/www/glpi/front/cron.php
```

A10.06 — Acompanhar o log do cron.

```
tail -n 30 /var/log/glpi/cron.log
```

Se não houver saída, valide as **Ações automáticas** na interface administrativa e o modo CLI da tarefa. Agende rotação do `cron.log` para evitar crescimento contínuo.

A11 — Requisitos de segurança para produção

A11.01 — HTTPS não é opcional para produção. O acesso por `http://192.168.18.150` é apenas uma configuração de laboratório. Para liberar usuários corporativos, configure um certificado confiável, um VirtualHost 443 com TLS e redirecionamento HTTP → HTTPS segundo a PKI e o nome de serviço corporativos. Esse processo depende de certificado, DNS e políticas da empresa; **não será inventada uma autoridade certificadora nem um hostname** neste manual.

A11.02 — Habilitar módulo SSL do Apache, quando houver certificado aprovado.

```
/usr/sbin/a2enmod ssl
```

A11.03 — Conferir validade do certificado escolhido. Substitua o caminho pelo arquivo real.

```
openssl x509 -in /CAMINHO/CERTIFICADO_APROVADO.pem -noout -subject -dates
```

A11.04 — Após habilitar HTTPS de ponta a ponta, localizar e ajustar no PHP-FPM:

```
session.cookie_secure = On
session.cookie_httponly = On
session.cookie_samesite = Lax
```

A11.05 — Reiniciar o FPM depois de salvar os ajustes.

```
systemctl restart "php${PHPVER}-fpm"
```

A11.06 — Verificar o cabeçalho HTTPS, quando estiver configurado. Substitua o nome pelo endereço corporativo real.

```
curl -I https://NOME_APROVADO_DA_EMPRESA
```

A11.07 — Segurança administrativa. Bloqueie contas padrão, use senhas longas, MFA se disponível/aprovada, restrição de rede, firewall, atualizações de segurança, usuário SQL específico, menor privilégio e contas nominativas. Configure as chaves criptográficas com permissões restritivas e inclua-as no backup seguro.

A11.08 — Segurança operacional. Não exponha MariaDB à internet. Não faça `chmod -R 777`. Não deixe `/var/www/glpi` inteiro gravável pelo serviço web. Instale plugins somente de fontes confiáveis e confirmadamente compatíveis com a versão 12.0.1.

A12 — Backup, restauração e monitoramento

A12.01 — Criar pasta protegida de backup local temporário. Backup local deve também ser replicado fora da VM; snapshot não substitui backup.

```
install -d -o root -g root -m 0700 /var/backups/glpi
```

A12.01a — Restringir permissões dos arquivos novos gerados nesta sessão.

```
umask 077
```

A12.01b — Fazer o Bash detectar falhas de qualquer comando do pipeline. Sem isso, uma falha no `mariadb-dump` poderia ser mascarada pela execução bem-sucedida do `gzip`.

```
set -o pipefail
```

A12.02 — Exportar o banco com compressão e privilégios administrativos locais.

```
mariadb-dump --single-transaction --routines --events glpidb | gzip > /var/backups/glpi/glpidb.sql.gz
```

A12.03 — Verificar se o dump não está vazio.

```
ls -lh /var/backups/glpi/glpidb.sql.gz
```

A12.04 — Validar integridade do arquivo comprimido.

```
gzip -t /var/backups/glpi/glpidb.sql.gz
```

A12.05 — Criar backup de configuração, dados, plugins e código. Essa cópia inclui segredos criptográficos; restrinja acesso e faça custódia adequada.

```
tar -czf /var/backups/glpi/glpi-arquivos.tar.gz \
  /etc/glpi /var/lib/glpi /var/log/glpi \
  /var/www/glpi /etc/apache2/sites-available/glpi.conf
```

A12.06 — Verificar o arquivo de backup.

```
tar -tzf /var/backups/glpi/glpi-arquivos.tar.gz | head
```

A12.07 — Conferir as permissões dos backups.

```
chmod 0600 /var/backups/glpi/glpidb.sql.gz /var/backups/glpi/glpi-arquivos.tar.gz
```

A12.08 — Teste de restauração controlado. Restaure **em VM separada de homologação**, sem sobrescrever a produção: banco, diretórios, chaves, permissões, plugins, PHP, web server e rede. Confirme login, chamado e download de anexo. Registre RPO/RTO real, duração e aceite. **Não há comprovação de recuperabilidade sem esse teste.**

A12.09 — Verificar logs e serviços diariamente/por monitoramento.

```
systemctl is-active apache2 mariadb "php${PHPVER}-fpm" cron
```

A12.10 — Consultar erros recentes do GLPI.

```
tail -n 40 /var/log/glpi/php-errors.log
```

A12.11 — Consultar erro do VirtualHost.

```
tail -n 40 /var/log/apache2/glpi_error.log
```

A12.12 — Monitoramento recomendado. Vigiar disco livre, vencimento TLS, tempo de resposta HTTP, status MariaDB/FPM/Apache, erros 500, crescimento da pasta de anexos, resultado dos backups, fila do cron e falhas de autenticação. Defina alertas e responsáveis antes da publicação.

TRILHA B — CORREÇÃO DO GLPI JÁ INSTALADO

B00 — Evidências reais do servidor e limites

No servidor `GLPI`, com acesso de teste a `192.168.18.150`, os registros fornecidos mostraram:

Evidência observada	Interpretação correta
Apache `active`	Serviço em execução; não comprova ausência de erro PHP
MariaDB `active`	Banco em execução; não comprova autenticação do GLPI
PHP-FPM `active`	Processo ativo; não comprova que o Apache está usando FPM
PHP CLI `8.4.27`	PHP atende ao requisito mínimo da família GLPI 12
`mkdir(): Permission denied`	O GLPI tentou criar pasta sem permissão suficiente
`Twig\\Error\\LoaderError` procurando `/var/lib/glpi/vendor/...`	Cache compilado estava referenciando caminho de bibliotecas que não correspondia à árvore de código
`system:check_requirements` era listado no console	Esse é o comando que funciona na versão mostrada
Após renomear e recriar `_cache`, a página voltou a funcionar	Recuperação do erro 500 **confirmada pelo operador**

> O trecho de log contendo `[mpm_prefork:notice]` e `[php:error]` sugere que `mod_php` esteve em uso em algum momento. Se a meta é PHP-FPM, verifique o módulo ativo na configuração **atual**, em vez de assumir que o serviço FPM ativo garante a integração.

B01 — Diagnóstico do erro 500 antes de alterar arquivos

B01.01 — Confirmar serviços.

```
systemctl is-active apache2 mariadb
```

B01.02 — Confirmar versão do PHP.

```
php -v
```

B01.03 — Conferir FPM correspondente.

```
PHPVER=$(php -r 'echo PHP_MAJOR_VERSION.".".PHP_MINOR_VERSION;')
```

```
systemctl is-active "php${PHPVER}-fpm"
```

B01.04 — Conferir erro Apache específico do GLPI.

```
tail -n 80 /var/log/apache2/glpi_error.log
```

B01.05 — Conferir log próprio da aplicação.

```
tail -n 80 /var/log/glpi/php-errors.log
```

B01.06 — Executar checagem de requisitos.

```
sudo -u www-data php /var/www/glpi/bin/console system:check_requirements
```

Critério: procure o erro mais recente, não apenas entradas antigas. Registre o horário antes de abrir a página e compare com o novo evento no log.

B02 — Corrigir `mkdir(): Permission denied` com segurança

O log original registrou `Safe\\Exceptions\\FilesystemException: mkdir(): Permission denied` em `SystemConfigurator.php`. Isso identifica problema de criação de diretório, mas **não nomeia a pasta específica**. Portanto, não se deve fazer `chown -R www-data` no código inteiro sem verificar os diretórios necessários.

B02.01 — Localizar os arquivos que definem os caminhos.

```
ls -l /var/www/glpi/inc/downstream.php /etc/glpi/local_define.php
```

B02.02 — Ler o redirecionamento de configuração.

```
cat /var/www/glpi/inc/downstream.php
```

B02.03 — Ler a definição das pastas externas.

```
cat /etc/glpi/local_define.php
```

B02.04 — Inspecionar proprietários e permissões.

```
ls -ld /etc/glpi /var/lib/glpi /var/lib/glpi/files /var/log/glpi
```

B02.05 — Testar permissão sobre a configuração efetiva.

```
sudo -u www-data test -w /etc/glpi && echo 'Configuracao gravavel'
```

B02.06 — Testar permissão sobre dados.

```
sudo -u www-data test -w /var/lib/glpi/files && echo 'Dados gravaveis'
```

B02.07 — Testar permissão sobre logs.

```
sudo -u www-data test -w /var/log/glpi && echo 'Logs gravaveis'
```

B02.08 — Somente se um desses diretórios externos estiver com o proprietário incorreto, corrija **a pasta afetada**, sem modificar o código. Exemplo para o diretório de dados:

```
chown -R www-data:www-data /var/lib/glpi/files
```

B02.09 — Para o diretório externo de configuração, quando estiver incorreto:

```
chown www-data:www-data /etc/glpi
```

B02.10 — Garantir permissão segura nesse diretório:

```
chmod 0750 /etc/glpi
```

B02.11 — Repetir o verificador de requisitos.

```
sudo -u www-data php /var/www/glpi/bin/console system:check_requirements
```

Critério: as permissões para arquivos de chave e diretórios devem aparecer `[OK]`. Não altere/regenere chaves criptográficas existentes para resolver um problema de permissão.

B03 — Correção validada: cache Twig com caminho incorreto

Sintoma real:

```
The "/var/lib/glpi/vendor/symfony/ux-twig-component/templates"
directory does not exist.
```

Por que ocorre? Os arquivos de cache compilados da aplicação referenciavam `vendor` a partir de `/var/lib/glpi`. O local do código e suas dependências é `/var/www/glpi/vendor`. No caso observado, **reconstruir o cache resolveu o problema**. Isso não justifica copiar bibliotecas para `/var/lib/glpi/vendor` ou criar um link simbólico para ocultar configuração incorreta.

B03.01 — Verificar a biblioteca no local correto.

```
ls -ld /var/www/glpi/vendor/symfony/ux-twig-component/templates
```

B03.02 — Conferir o diretório do cache.

```
ls -ld /var/lib/glpi/files/_cache
```

B03.03 — Agendar janela de manutenção. Notifique os usuários e confirme que não há atualização ou importação em andamento. No caso de CRON ativo, **suspenda somente a tarefa do GLPI** ou execute a mudança em uma janela sem execução concorrente. Em VM compartilhada não desligue globalmente `cron` sem avaliação.

B03.04 — Parar o Apache durante a regeneração.

```
systemctl stop apache2
```

B03.05 — Entrar no diretório de dados externo.

```
cd /var/lib/glpi/files
```

B03.06 — Renomear o cache em vez de excluí-lo. Esse comando só deve ser executado se `_cache` existir, após confirmar que a aplicação está em manutenção.

```
mv _cache "_cache.backup.$(date +%Y%m%d-%H%M%S)"
```

B03.07 — Recriar cache vazio e gravável por `www-data`.

```
install -d -o www-data -g www-data -m 0750 /var/lib/glpi/files/_cache
```

B03.08 — Conferir dono/permissões.

```
ls -ld /var/lib/glpi/files/_cache
```

Resultado esperado: `www-data www-data` e permissões equivalentes a `drwxr-x---`.

B03.09 — Iniciar Apache novamente.

```
systemctl start apache2
```

B03.10 — Confirmar serviço.

```
systemctl is-active apache2
```

B03.11 — Testar os requisitos após reconstrução.

```
sudo -u www-data php /var/www/glpi/bin/console system:check_requirements
```

B03.12 — Testar acesso HTTP em homologação.

```
curl -I http://192.168.18.150
```

B03.13 — Conferir se há novo erro de Twig.

```
tail -n 40 /var/log/glpi/php-errors.log
```

Critério: a tela do GLPI carrega sem erro HTTP 500 e não há nova falha de carregamento de templates. **Esse resultado foi informado como alcançado no seu ambiente.**

Se o erro persistir: pare. Inspecione novamente `downstream.php` e `local_define.php`; verifique a versão dos arquivos `vendor`; confira novos logs; não crie links simbólicos compensatórios e não sobrescreva a instalação.

Rollback apenas se autorizado: mantenha o diretório `_cache.backup.DATA` para investigação; normalmente não se deve restaurar cache compilado incorreto. Em caso de instabilidade, restaure do ponto de recuperação validado/backup da mudança, sem mexer no banco por tentativa.

B04 — Evitar as falhas de comandos encontradas

Mensagem ou situação	Explicação	Procedimento correto
`a2enmod: command not found`	`/usr/sbin` ausente no PATH	Usar `/usr/sbin/a2enmod ...` ou ajustar PATH
`/usr/sbin/a2enmod.: No such file`	Ponto indevido no final	Remover o ponto
`Which module(s) do you want to enable?`	`a2enmod` executado sem argumentos	`Ctrl+C` e executar com os nomes
`a2enconf: command not found`	Mesmo problema de PATH	`/usr/sbin/a2enconf "php${PHPVER}-fpm"`
`There are no commands defined in glpi:system`	Namespace não existe nessa versão	`system:check_requirements`
`Permissions for security keys files [ERROR]`	Conta web não grava onde estão as chaves	Inspecionar diretório real e ajustar menor permissão possível
`Twig LoaderError` com `/var/lib/glpi/vendor`	Referência incorreta de cache compilado	Reconstruir `_cache` em manutenção, como B03
PHP-FPM `active`, mas requisições PHP falham	Serviço pode não estar ligado ao Apache	Conferir `a2enconf`, módulos, vhost e logs
`AH00558 ServerName`	Aviso de FQDN do Apache	Não é, isoladamente, erro 500
`php-imap` sem candidato	Pacote não disponível no repositório	Remover do conjunto básico; avaliar necessidade futura

B04.01 — Verificar módulo PHP clássico.

```
/usr/sbin/apache2ctl -M | grep php_module
```

B04.02 — Verificar o proxy FPM.

```
/usr/sbin/apache2ctl -M | grep proxy_fcgi
```

B04.03 — Verificar a configuração FPM habilitada.

```
ls -l "/etc/apache2/conf-enabled/php${PHPVER}-fpm.conf"
```

B04.04 — Validar sintaxe do Apache.

```
/usr/sbin/apache2ctl configtest
```

Não faça a troca entre mod_php e PHP-FPM em servidor com vários sites sem mapear as dependências de todos eles.

TRILHA C — HOMOLOGAÇÃO, OPERAÇÃO E ACEITE

C01 — Testes funcionais e de recuperação

Abra o GLPI pelo endereço aprovado e execute os seguintes testes com contas **de teste**, sem expor dados pessoais ou corporativos reais desnecessariamente.

ID	Teste	Resultado esperado	Status/assinatura
T01	Apache, MariaDB, PHP-FPM ativos	Todos `active`	_____
T02	Página de login acessível	Sem HTTP 500	_____
T03	Login administrador nominal	Sessão autenticada; padrão alterado	_____
T04	Verificação CLI de requisitos	Nenhum `[REQUIRED] [ERROR]`	_____
T05	Criar chamado de teste	Número gerado e persistência	_____
T06	Reabrir/atualizar chamado	Histórico e status corretos	_____
T07	Anexar e baixar arquivo inofensivo	Integridade e permissão corretas	_____
T08	Conta de técnico	Visualiza e atende conforme perfil	_____
T09	Conta de solicitante	Acesso limitado ao próprio escopo	_____
T10	Cron executa sem erro novo	Ações automáticas funcionais	_____
T11	HTTPS e cookie `Secure` em produção	Certificado válido e navegação TLS	_____
T12	Backup SQL + dados	Arquivos gerados e íntegros	_____
T13	Restauração em ambiente isolado	Login/chamados/anexos recuperados	_____
T14	Monitoração	Alertas e responsáveis definidos	_____
T15	Plugins (quando houver)	Compatibilidade e tarefas validadas	_____
T16	Notificações/SMTP (quando em escopo)	Entrega e fila confirmadas	_____
T17	Inventário GLPI Agent (quando em escopo)	Agente autorizado e item recebido	_____
T18	Logs após navegação	Sem novos erros críticos	_____

Nota: T16/T17 são condicionais aos serviços habilitados; se estiverem fora do escopo, registre formalmente **N/A**, não “aprovado”. *A ausência de erro 500 não equivale a 100% desses testes.*

C02 — Checkpoints GO / NO-GO

GO (liberação) somente se os itens críticos T01–T14 aplicáveis estiverem aprovados, o administrador estiver configurado, HTTPS estiver operacional, a restauração em VM isolada tiver sido bem-sucedida, backups forem protegidos, não houver erros críticos recentes, a mudança tiver aprovação e houver responsável pelo monitoramento.

NO-GO (não liberar) se houver erro 500 recorrente; PHP/Apache sem verificação do mecanismo de execução; falha de chaves; banco exposto à rede; conta admin com senha padrão; backups sem teste de restauração; HTTP desprotegido em acesso corporativo; incompatibilidade de plugin crítico; ou alteração sem rollback.

C03 — Registro de execução

- **SO / versão:** _____
- **GLPI / versão:** _____

- **Host/IP:** _____
- **MariaDB / versão:** _____
- **PHP / versão:** _____
- **Apache / versão:** _____
- **Diretório de código:** _____
- **Diretório de configuração:** _____
- **Diretório de dados/cache:** _____
- **Backup (destino e data):** _____
- **Restauração de teste (data):** _____
- **Mudança/CHG:** _____
- **Executor / validador:** _____
- **Resultado:** APROVADO / REPROVADO / PENDENTE

C04 — Referências oficiais e notas de controle

1. **GLPI Project, 28/10/2026 — versões 12.0.1 / 11.0.12 e segurança:** https://www.glpi-project.org/en/two-new-glpi-versions-are-available-2/
2. **Requisitos GLPI 12 e compatibilidade de plugins:** https://www.glpi-project.org/en/glpi-12-release/
3. **Arquivos oficiais de distribuição:** https://www.glpi-project.org/en/downloads/
4. **Instalação e separação de diretórios:** https://glpi-install.readthedocs.io/en/latest/install/
5. **Apache, PHP e requisitos:** https://glpi-install.readthedocs.io/en/latest/prerequisites.html
6. **Ferramentas CLI do GLPI:** https://glpi-install.readthedocs.io/en/latest/command-line.html
7. **Tarefas automáticas:** https://help.glpi-project.org/documentation/modules/configuration/crontasks

Nota de responsabilidade: os erros e a solução de cache acima refletem os logs fornecidos e a confirmação do usuário de que o sistema voltou a funcionar. O restante deste manual é um procedimento técnico proposto, **não uma auditoria remota do ambiente**. Antes de implantar em produção, validar a compatibilidade da distribuição/versão de PHP, certificar o download, revisar a segurança e concluir testes de homologação.